EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

Department of Data Science and Engineering

Advance techniques in Aspect Based Sentiment Analysis

Supervisor:

Md Easin Arafat PhD.

Phd Student

Author:

Abdul Basit

Computer Science BSc

Budapest, 2025

EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

Thesis Topic Registration

Form Student's Data:

Student's Name: *Abdul Basit*
Student's Neptun code: *EE8LMT*

Educational Information:

Training programme: *Computer Science BSc*

I have an external supervisor

Internal Supervisor's Name: ***Md. Easin Arafat***
Supervisor's Home Institution: ***Department of Data Science and Engineering***
Address of Supervisor's Home Institution: ***1117, Budapest, Pázmány Péter sétány 1/C.***
Supervisor's Position and Degree: **PhD Student**

External Supervisor's Name: ***Muhammad Zain Sheikh***
Supervisor's Home Institution: ***Debreceni Egyetem Informatikai Kar***
Address of Supervisor's Home Institution: ***Debrecen, Kassai út 26., 4028***
Supervisor's Position and Degree: ***BSc Computer Science Engineering, DevOps Engineer at Ericsson***
Supervisor's e-mail address: ***zainsheikh64@gmail.com***

Thesis Title: Advanced Techniques in Aspect-Based Sentiment Analysis
Topic of the Thesis:
(Upon consulting with your supervisor, give a 150-300-word-long synopsis of your planned thesis.)

Main Idea:

Using machine learning models to accurately identify and analyze sentiments related to specific features of products or services. Importance: Enhances precision in sentiment analysis, improving product insights and customer feedback analysis.

Applications:

- E-commerce: Detailed product reviews.
- Services: Customer feedback on specific service attributes.

Algorithms:

Sentiment classification, Natural Language Processing (NLP), deep learning (e.g., BERT, GPT), SVMs (Support Vector Machines) Tools: Python, NLTK, spaCy, TensorFlow, PyTorch, Scikit-learn, Matplotlib, Seaborn.

Dataset:

Online review datasets (kaggle), real time data from review platforms like Amazon, Google Review: Model

Performance Evaluation:

Accuracy, precision, and F1-score to evaluate sentiment classification effectiveness.

Outcome:

A robust system capable of accurately predicting and categorizing customers sentiments related to distinct features of a product or service.

Budapest, 2025. 03. 26.

Contents

The project must be evaluated by internal and external supervisors. (30-point evaluation paper).

This project is deployed on the following domain:

https://sentimeter.space/

Chapter 1
1. Introduction

1.1 Motivation

In the contemporary digital landscape, understanding user sentiment has become an essential factor for businesses seeking sustainable growth and competitive advantage. With the rapid expansion of online platforms, social media channels, and customer review websites, the volume of available textual data—containing valuable insights into customer opinions—has grown exponentially. Businesses, educational institutions, and individual content creators constantly receive feedback in the form of user-generated texts, such as customer reviews, comments, and survey responses. Analysing this vast amount of textual feedback manually is not only impractical but also inefficient and error-prone, often leading to incomplete or biased interpretations.

Traditional sentiment analysis approaches often oversimplify the complexities inherent in human language. They generally classify user-generated content broadly into positive, negative, or neutral categories. However, even this simpler approach poses significant challenges due to nuances such as sarcasm, irony, mixed sentiments, and varying levels of sentiment intensity within natural language. Accurately interpreting these subtleties is vital, as misunderstandings or misinterpretations can lead businesses to incorrect decisions, negatively impacting customer satisfaction, marketing strategies, and overall brand perception.

Motivated by these significant challenges, this project seeks to develop a robust and intelligent sentiment analysis platform capable of handling diverse textual inputs, thereby providing businesses and individuals with reliable, accurate, and actionable sentiment insights. This platform leverages state-of-the-art machine learning models, specifically fine-tuned NLP transformers such as DeBERTa, to capture the complex nuances of human language effectively. Furthermore, the system combines these advanced NLP techniques with intuitive visualization tools and user-friendly interfaces to empower users to clearly and quickly interpret sentiment results without extensive technical expertise.

By incorporating modern web technologies—such as a responsive frontend built using React and Next.js, coupled with an efficient backend developed with FastAPI—the system ensures scalability, ease of use, and real-time analysis capabilities. The comprehensive sentiment analysis functionality supports direct text inputs, batch CSV uploads, and extraction of comments from online platforms like YouTube. Thus, the core motivation behind this project is to bridge the gap between sophisticated natural language processing capabilities and accessible, practical sentiment analysis tools, enabling users to gain meaningful insights and respond effectively to customer sentiments.

1.2 Goals

The primary objective of this project is to create a reliable, intuitive, and scalable sentiment analysis platform tailored to handle various textual inputs effectively. By combining advanced

NLP models and modern software development practices, the project aims to achieve the following detailed goals:

1. **Implement Advanced NLP Models:**

   Develop and deploy a highly accurate sentiment classification model utilizing state-of-the-art transformer-based NLP models, specifically the DeBERTa architecture, to achieve superior accuracy and contextual understanding compared to traditional methods.

2. **Comprehensive Text Sentiment Classification:**

   Provide accurate sentiment categorization into clear, actionable categories—positive, negative, and neutral—ensuring nuanced interpretations across diverse user-generated content.

3. **Flexible Input Methods:**

   Allow users to submit text for analysis through multiple convenient methods, including:

   - Direct text input via the user interface for immediate sentiment feedback.
   - CSV file uploads to efficiently analyse large volumes of textual data.
   - Automated extraction and sentiment analysis of user comments from YouTube videos.

4. **Real-Time Analysis Capabilities:**

   Enable instant or near-instant sentiment analysis, allowing businesses and individuals to react promptly and strategically to customer feedback or emerging sentiment trends.

5. **User-Centric Design and Interface:**

   Design a visually appealing, intuitive, and responsive web interface using React, Tailwind CSS, and Framer Motion, ensuring the sentiment analysis results are easily understandable and accessible even to non-technical users.

6. **Scalable and Reliable Backend Infrastructure:**

   Build a robust, efficient, and maintainable backend system using FastAPI, PyTorch, and Hugging Face Transformers, ensuring efficient processing of data and smooth operation under increased load conditions.

7. **Comprehensive Testing and Quality Assurance:**

   Ensure system reliability and accuracy through rigorous testing practices, including backend unit testing with Pytest and thorough API endpoint testing with Postman.

8. **Detailed and Accessible Documentation:** Provide extensive, detailed, and clearly structured documentation that guides both end-users and developers, covering all essential aspects from installation, configuration, and system use to technical details and maintenance guidelines.

By accomplishing these goals, the project will deliver a powerful, practical, and easy-to-use sentiment analysis solution, enabling users to effectively harness the insights available in large textual datasets.

1.3 Scope of the Project

This Sentiment Analysis project is specifically designed to provide a robust and comprehensive solution for businesses, educational institutions, content creators, and individual users who require precise sentiment insights from textual feedback. The clearly defined project scope encompasses the following core functionalities and components:

1. **Advanced Sentiment Classification:**
    - Accurate categorization of textual content into positive, negative, or neutral sentiment classes.
    - Utilization of DeBERTa Transformer models, ensuring high classification accuracy, even with complex linguistic expressions such as sarcasm, irony, and mixed sentiment.
2. **Multiple Data Input Options:**
    - Direct textual input through the interactive user interface.
    - CSV bulk data upload capability, facilitating the efficient processing of large datasets.
    - Integration with the YouTube API for automated sentiment analysis of video comments.
3. **Intuitive User Interface:**
    - A modern, responsive, and intuitive frontend designed with React.js and Next.js, optimized for usability across desktops, tablets, and mobile devices.
    - Clear, interactive visualizations of sentiment analysis results, including real-time graphical representations such as charts and progress bars.
4. **Real-Time Processing and Feedback:**
    - Quick, efficient sentiment analysis enabling instant user feedback.
    - Real-time results update, providing immediate insights and responsiveness.
5. **Scalable Backend Infrastructure:**

- Backend implementation using FastAPI for robust and efficient API management.
- Comprehensive management of resources and computational efficiency using Docker containers and cloud deployment strategies (e.g., AWS, Azure).

6. **User Authentication and Data Security:**
   - Secure user registration, authentication, and profile management, safeguarding sensitive data and analysis history.
   - Implementation of industry-standard security measures to ensure data privacy and compliance with data protection regulations.
7. **Comprehensive Testing Strategy:**
   - Detailed backend unit testing with Pytest to ensure system reliability and accuracy of results.
   - API endpoint testing using Postman for robust functionality verification and consistent performance under diverse conditions.
8. **Documentation and Knowledge Transfer:**
   - Comprehensive user documentation providing clear setup instructions, system requirements, and usage guidelines.
   - Detailed developer documentation including code explanations, API specifications, environment setups, and deployment guidelines.

By strictly adhering to this well-defined scope, the Sentiment Analysis system aims to deliver a valuable and effective analytical tool, empowering users to effectively interpret, understand, and respond to customer and user sentiment, thereby significantly enhancing decision-making processes and user engagement.

Chapter 2

2. User Documentation

2.1 Introduction

Welcome to the Sentiment Analysis Dashboard. This innovative web application is specially designed to help users swiftly and accurately analyze the sentiment of textual data. Whether analyzing customer reviews, social media comments, survey responses, or user-generated content from various platforms, this platform provides intuitive, comprehensive sentiment analysis insights.

The Sentiment Analysis Dashboard simplifies the complex task of interpreting large volumes of textual data by offering clear, actionable insights into the sentiment expressed. Leveraging advanced machine learning models (DeBERTa transformer models), this tool ensures that sentiment analysis results are accurate, insightful, and easily understandable, enabling informed and strategic decision-making.

2.2 Used Methods and Tools

The system employs state-of-the-art technologies, methods, and tools to ensure robust performance, accurate sentiment classification, and a seamless user experience. Below is a detailed list of the primary tools and methods integrated into the system:

1. **Frontend Technologies:**
    1. React.js – Provides an efficient and interactive frontend experience.
    2. Next.js – Enables optimized performance, SEO capabilities, and server-side rendering.
    3. Tailwind CSS – Ensures modern, responsive, and visually appealing designs.
    4. Framer Motion – Adds smooth animations and interactive user experiences.
    5. Recharts – Visualizes sentiment results clearly through interactive graphs and charts.
2. **Backend Technologies:**
    1. FastAPI – Offers fast, reliable, and scalable backend API functionality.
    2. PyTorch – Facilitates deep learning capabilities.
    3. Hugging Face Transformers (DeBERTa) – Provides powerful natural language processing capabilities to accurately classify sentiments.
3. **Development & Deployment Tools:**

1. Docker – Ensures consistent deployment across development, testing, and production environments.
2. Git & GitHub – Supports efficient source code management, version control, and collaborative workflows.
3. Postman – Enables effective API testing and validation.
4. Visual Studio Code – An advanced, versatile code editor supporting efficient development practices.

2.3 User Guide

2.3.1 Hardware Requirements

The Sentiment Analysis web application is compatible with systems meeting the following hardware configurations:

Component	Minimum	Recommended
Processor	2.0 GHz dual-core CPU	3.5 GHz or faster quad-core CPU
Memory	4 GB RAM	8 GB RAM or higher
Display	1024x768 resolution (SVGA)	1920x1080 resolution (Full HD)
Storage	2 GB available disk space	SSD recommended

Table 1

Performance issues may occur on devices below these minimum requirements.

2.3.2 Network Requirements

For optimal user experience, your network should satisfy:

- **Bandwidth:** 50 KBps (400 kbps) or higher
- **Latency:** Under 150 ms

2.3.3 Supported Web Browsers

The platform is compatible with the following browsers:

- Microsoft Edge (latest version)
- Mozilla Firefox (latest version)
- Google Chrome (latest version)

2.4 Installation and Configuration

Prerequisites

Before installation, ensure your system includes:

- **Git:** For version control and repository management
- **Python (3.9 or higher):** Required for running backend services

Installing Git

1. Download Git from https://git-scm.com/downloads
2. Follow on-screen instructions for your operating system.

Installing Python

1. Visit https://www.python.org/downloads/ to install Python (3.9 or later).
2. Verify installation via command prompt:

```
python –version
```

Setup Instructions

Cloning the Repository

1. Open terminal or command prompt.
2. Navigate to your preferred directory:

```
cd ~/your-directory
```

3. Clone repository:

```
git clone https://github.com/Malik22G/Thesis.git
cd sentiment-analysis-frontend
```

Setting Up the Environment

1. Create a virtual environment:

```
python -m venv venv
```

2. Activate virtual environment:

- **Windows:**

```
venv\Scripts\activate
```

- **Mac/Linux:**

```
source venv/bin/activate
```

3. Install dependencies:

```
cd Backend/API
pip install -r requirements.txt
```

Running the Application

1. Ensure virtual environment activation.
2. Navigate to project folder.
3. Run backend server (FastAPI):

```
uvicorn app:app --reload
```

4. Run frontend application:

Open a new terminal and in sentiment-analysis-frontend directory run the following commands:

```
npm install
```

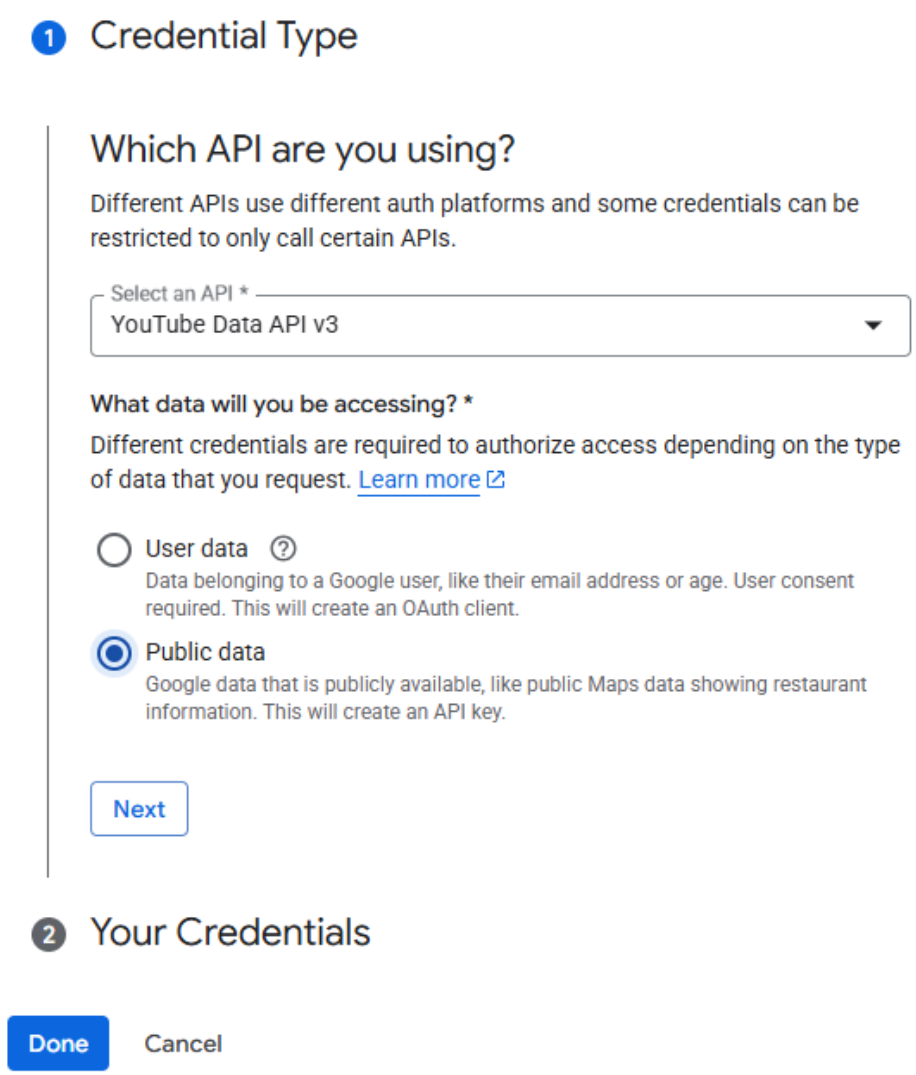
or if you get error due to version conflicts you can try

```
npm install --legacy-peer-deps
```

Before running the application frontend please make sure you have set up the envirvorment variable in the .env.local file you can do this by creating a .env.local file in the sentiment-analysis-frontend folder and then set the following two environment variables

```
NEXT_PUBLIC_YOUTUBE_API_KEY=your_youtube_api_key
NEXT_PUBLIC_BACKEND_URL="http://localhost:8000"
```

For creating your youtube api key visit https://console.cloud.google.com/apis and create a new project and activate YouTube Data API v3 and after activation click on create credentials



After creating the key add it to your .env.local file as

```
NEXT_PUBLIC_YOUTUBE_API_KEY=your_youtube_api_key
```

Then you can run the project locally with the following commands

```
npm run build
npm start
```

Open the application in your browser:

- Frontend: http://localhost:3000
- Backend API: http://localhost:8000/docs

2.5 Features of the Sentiment Analysis System

The Sentiment Analysis application includes the following key features:

1. Text-Based Sentiment Analysis

- **Functionality:**

  Analyse sentiment from direct textual inputs, providing immediate insights.

- **Usage:**

1. Go to the "Text Analysis" section.
2. Enter the text you wish to analyse.
3. Click "Analyse Sentiment" to receive results, displayed visually.

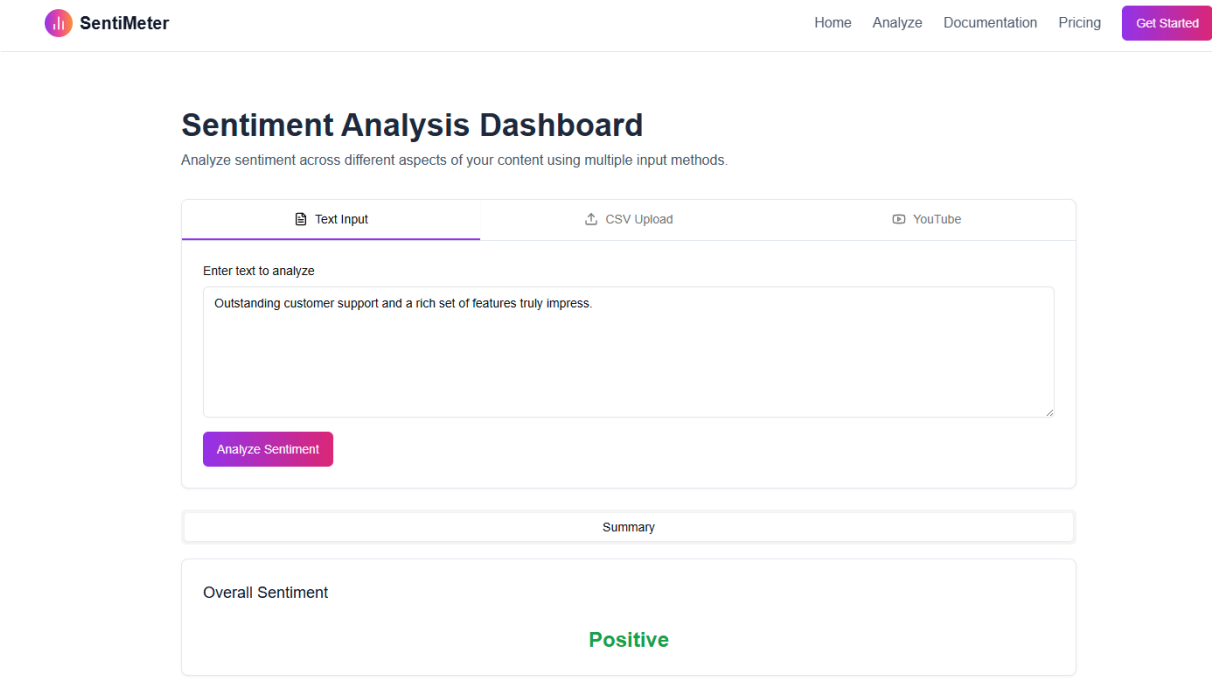


Figure 1: Single text Input (Positive)

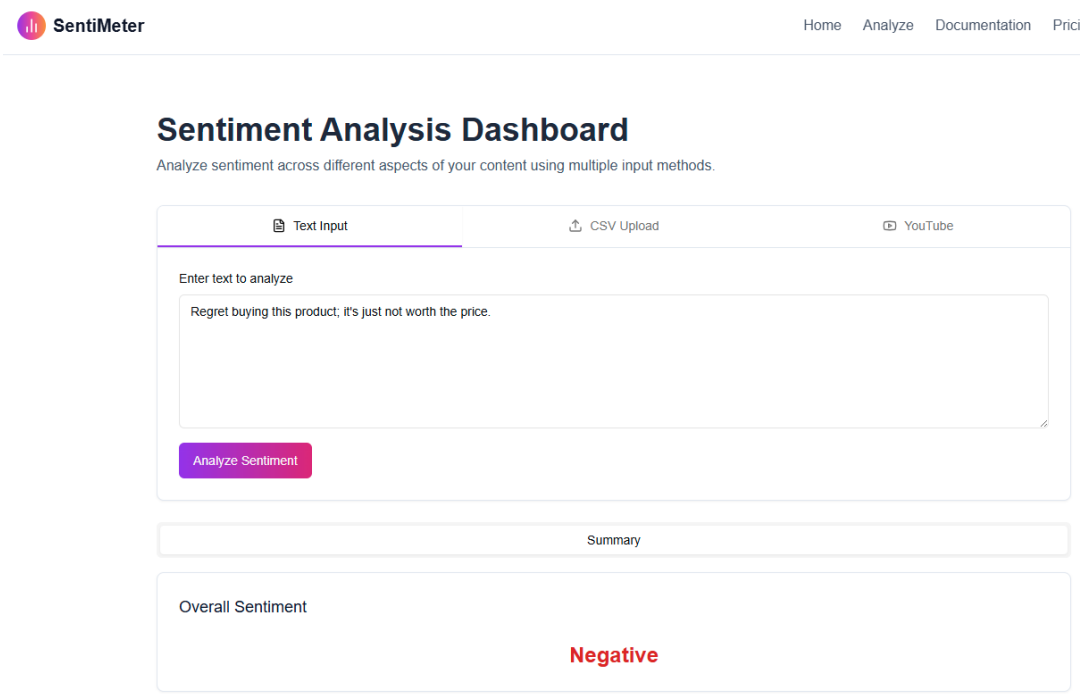


Figure 2: Single text Input (Negative)

2. CSV Bulk Sentiment Analysis

- **Functionality:**
  Process large-scale sentiment analysis from CSV files efficiently.
- **Usage:**

1. Navigate to "CSV Upload" section.
2. Upload your CSV file containing textual data.
3. The system processes the file, presenting sentiment results clearly.

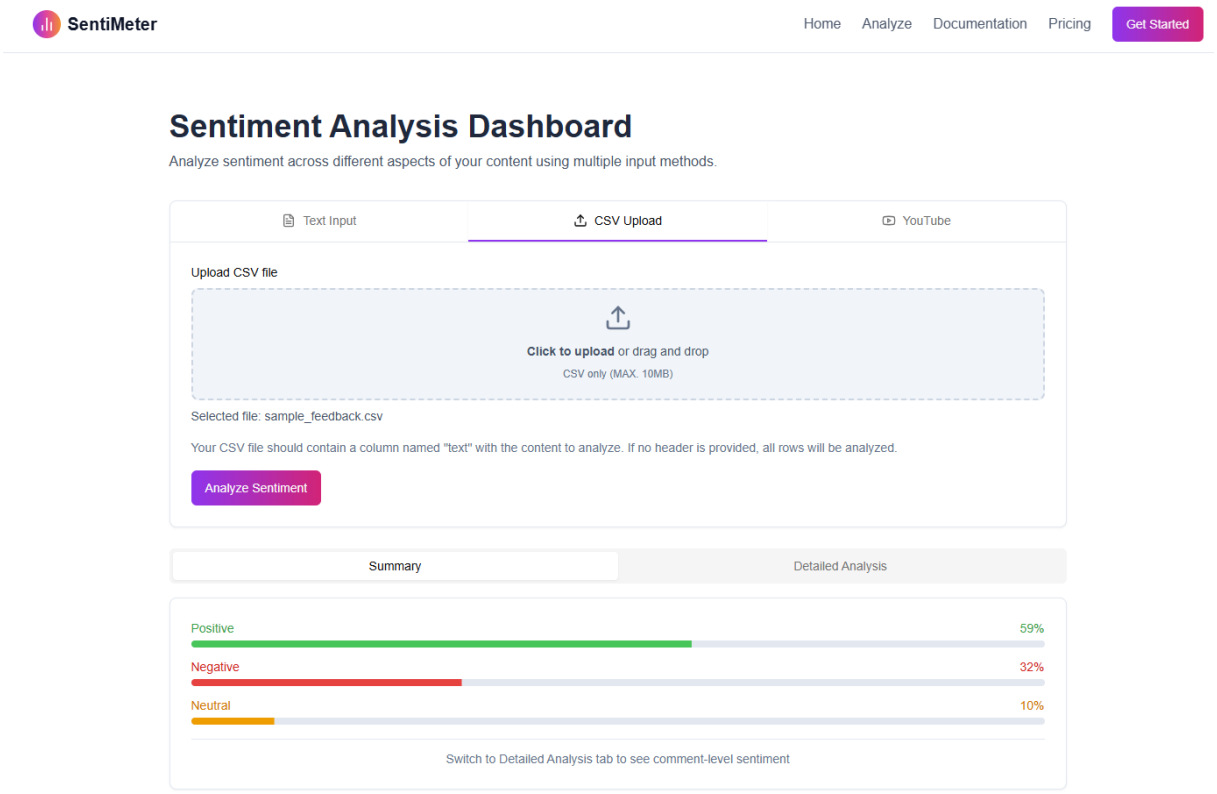


Figure 3: CSV file upload

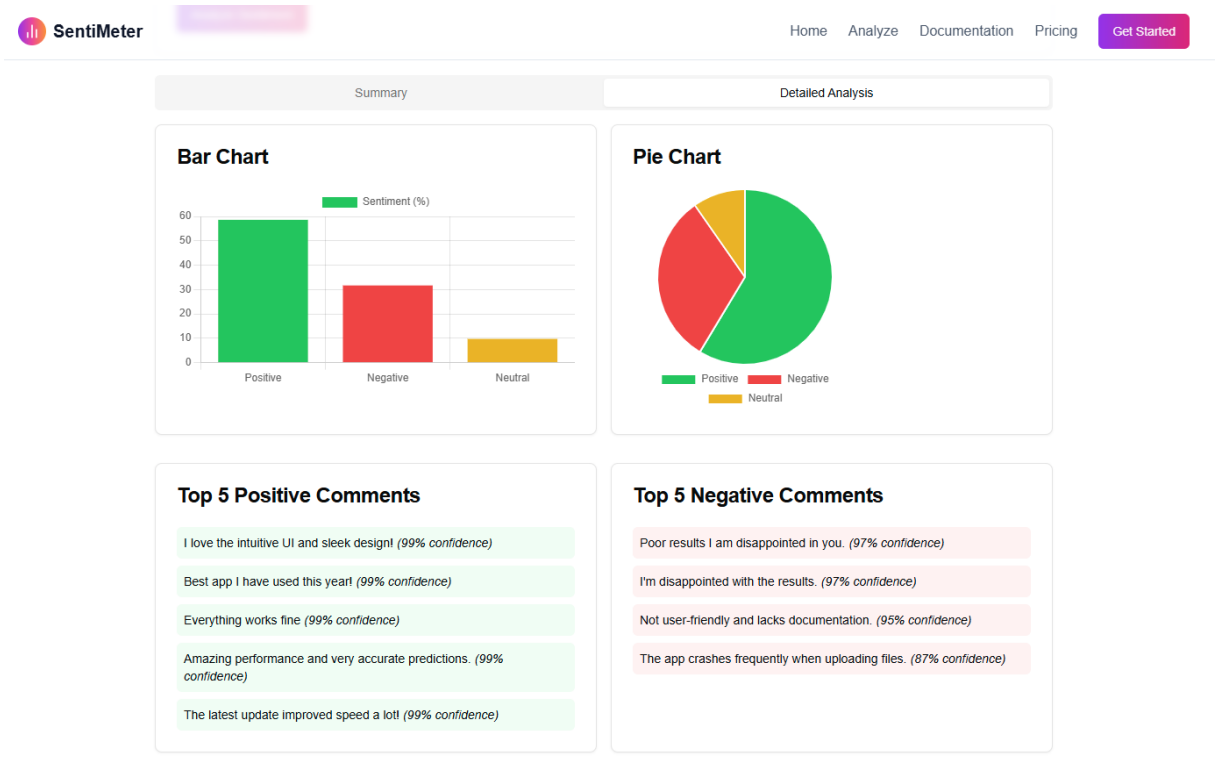


Figure 4: Detailed analysis tab

3. YouTube Comment Sentiment Analysis

- **Functionality:**
  Analyse user comments from YouTube videos.
- **Usage:**

1. Access "YouTube Analysis" section.
2. Input the YouTube video URL.
3. Click "Analyse Sentiment" to display detailed sentiment analysis of video comments.

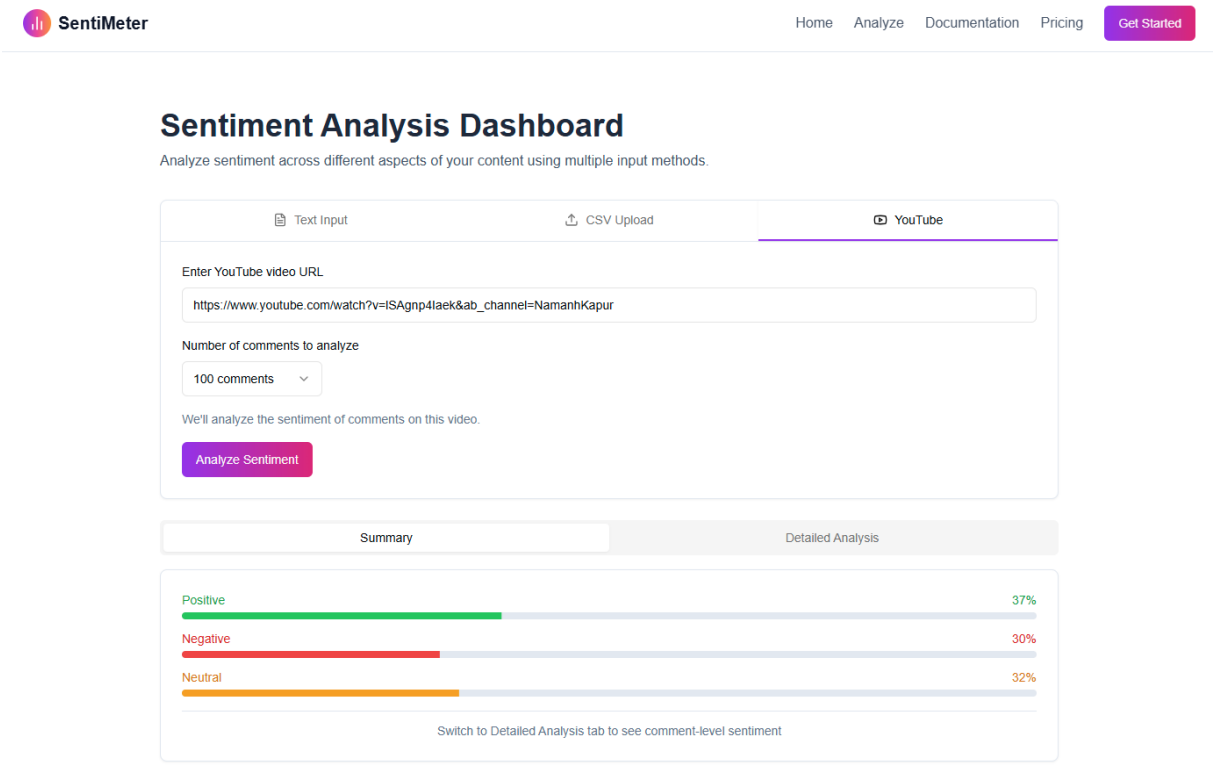


Figure 5: YouTube link to fetch comments and analyse

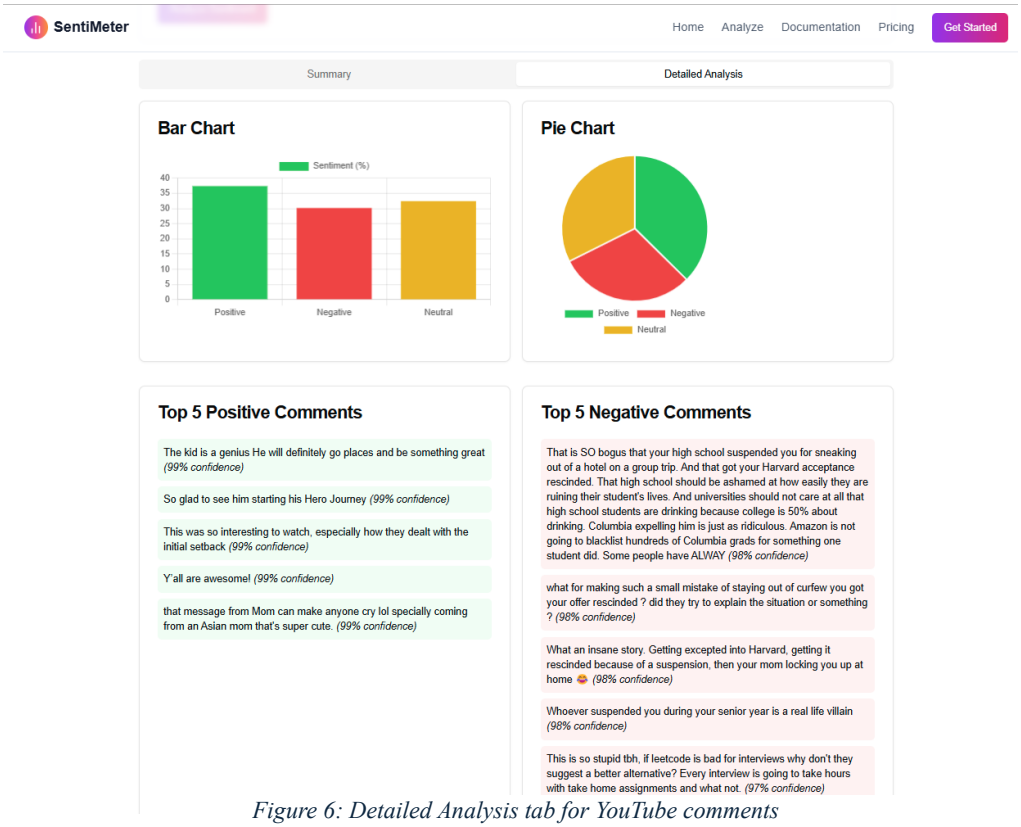
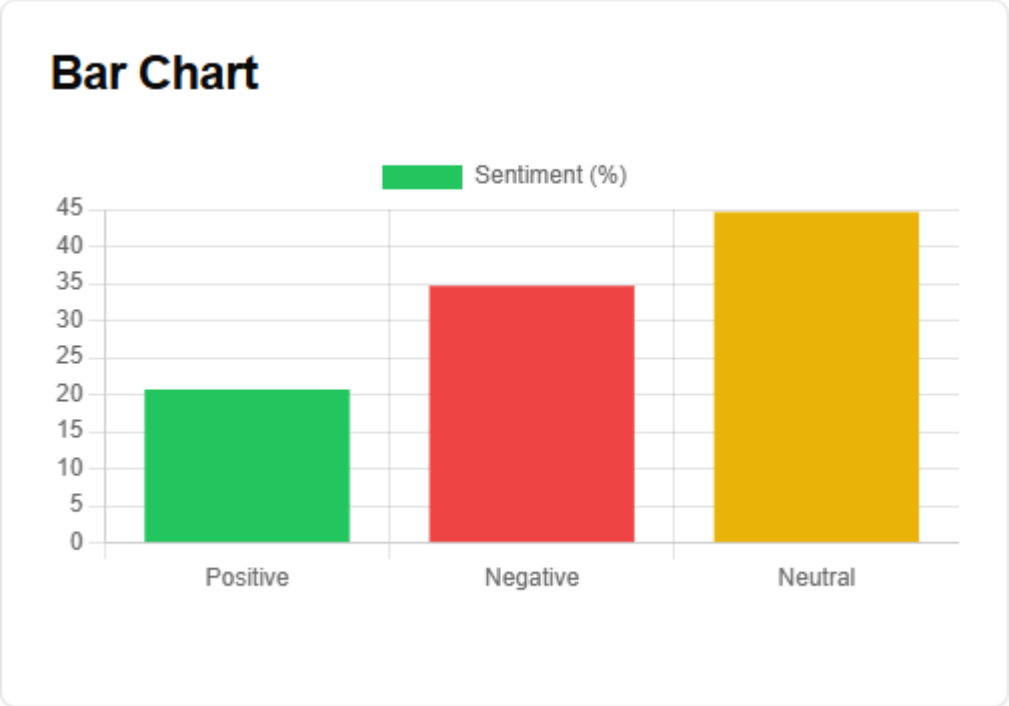


Figure 6: Detailed Analysis tab for YouTube comments

4. Interactive Visual Reports

- **Functionality:**
  Visualize sentiment data through clear, interactive charts.
- **Usage:** Sentiment analysis results automatically display using bar charts, pie charts, and interactive visual summaries, providing quick, easy interpretation.



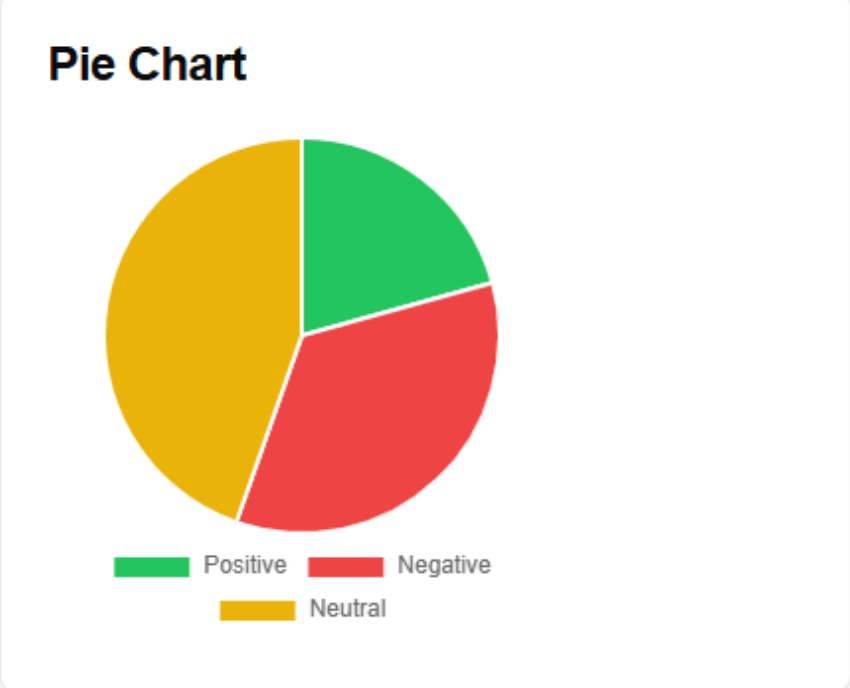


Figure 7: Bar Chart and Pie Chart

5. Real-Time Sentiment Insights

- **Functionality:**
  Instantly receive sentiment analysis feedback.
- **Usage:**
  Input data via any available method; analysis results immediately displayed, enabling rapid decision-making.

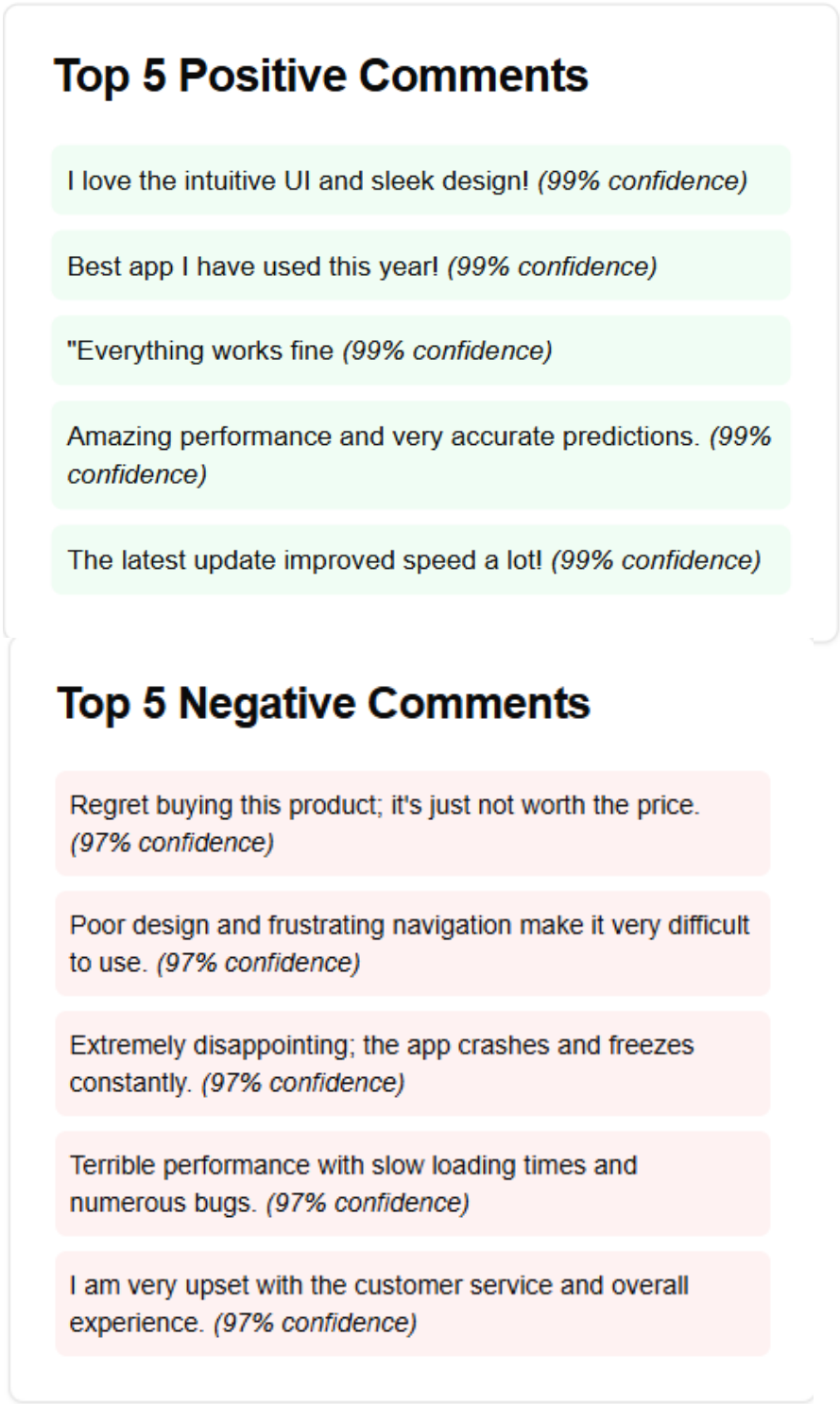


Figure 8: Top negative and positive Comments

Troubleshooting and FAQs

- **Q:** My analysis results are not displaying correctly.

  A: Check network connection, ensure backend and frontend servers run correctly, and verify environment variables.

- **Q:** "ModuleNotFoundError" displayed during startup.

A: Run `pip install -r requirements.txt` again and ensure the virtual environment is activated.

- **Q:** API keys not working or invalid.

A: Ensure `.env` file correctly contains your API keys. Double-check spelling and formatting.

Additional Resources

- **FastAPI Documentation:** https://fastapi.tiangolo.com
- **React Documentation:** https://reactjs.org
- **Hugging Face Transformers:** https://huggingface.co/docs/transformers
- **Docker:** https://docs.docker.com
- **PyTorch:** https://pytorch.org/docs

Chapter 3

3. Developer Documentation

3.1 Problem Specification

The Sentiment Analysis Dashboard is a web application designed to perform efficient and accurate sentiment classification of textual data provided by users. Sentiment analysis has become vital in various sectors for understanding consumer opinions, market trends, public perceptions, and user experiences. Businesses rely heavily on the accurate interpretation of textual feedback to drive strategic decisions, improve customer satisfaction, and refine product offerings.

This project specifically addresses the following core problems:

1. **Efficient Sentiment Classification:**
   Accurately classify textual content into predefined sentiment categories: positive, negative, and neutral.
2. **Multiple Input Sources:**
   Support sentiment analysis from diverse sources such as direct text input, batch analysis through CSV uploads, and automated analysis of YouTube video comments.
3. **Real-Time Sentiment Analysis:**
   Provide fast, near-instantaneous sentiment analysis results to enable timely decision-making for businesses and content creators.
4. **Intuitive Visualization:**
   Present sentiment analysis results through clear, easy-to-understand visualizations and interactive user interfaces.

To address these challenges, the system integrates powerful NLP transformer-based models (DeBERTa) known for their high accuracy in sentiment classification and advanced contextual language understanding.

3.2 Tools and Methods Used

The following comprehensive methods, tools, and approaches were integrated to construct the Sentiment Analysis Dashboard:

3.2.1 Machine Learning and NLP Tools

1. **Hugging Face Transformers (DeBERTa Model):**

   Pretrained DeBERTa model fine-tuned for sentiment classification.

   Provides high accuracy and robustness, effectively capturing nuanced linguistic patterns.

2. **PyTorch:**

   Framework supporting deep learning functionalities and efficient computation. Ensures scalable, robust model training and inference.

3.2.2 APIs and Frameworks

1. **FastAPI:**

   Fast, lightweight Python framework to build robust REST APIs.

   Handles HTTP requests, routing, and efficient data validation through Pydantic.

2. **YouTube Data API (Google API):**

   Extracts YouTube comments for sentiment analysis.

3.2.3 Frontend Technologies

1. **React.js:**

   Component-based UI design ensuring interactive, responsive user experience.

2. **Next.js:**

   Optimizes frontend performance, providing server-side rendering (SSR) capabilities.

3. **Tailwind CSS:**

   Modern, responsive CSS framework ensuring consistent styling and rapid UI development.

4. **Framer Motion:**

   Adds intuitive animations and interactive user experiences.

5. **Recharts:**

   Visualizes sentiment analysis results clearly through interactive charts and graphs.

3.2.4 Development Environment and Tools

1. **Visual Studio Code (VS Code):**

   Integrated development environment (IDE) supporting efficient coding practices and debugging.

   Extensions include Python, Docker, Prettier for formatting.

2. **Git and GitHub:**

   Version control and collaborative project management.

3.2.5 Testing and Debugging

1. **Pytest:**

   Python testing framework ensuring robust backend functionality through unit testing.

2. **Postman:**

   API endpoint testing, ensuring correct functionality and performance.

3. **Browser Developer Tools:**

   Frontend debugging, ensuring a responsive user experience.

3.3 Key Functionalities

The following functionalities constitute the core features of the Sentiment Analysis Dashboard:

1. **Sentiment Classification:**

   Accurately classifies textual inputs into positive, negative, and neutral categories using DeBERTa NLP models.

2. **Multi-Input Handling:**

   Direct textual input via interactive forms.

   CSV file uploads for bulk sentiment analysis.

   YouTube comment analysis through integration with the YouTube Data API.

3. **Real-Time Results:**

   Provides instant feedback and real-time analysis results using efficient backend inference.

4. **Data Visualization:**

   Interactive and intuitive visualizations using bar charts, pie charts, and sentiment distribution graphs.

3.4 Backend Structure and API Endpoints

FastAPI Endpoints:

1. **Sentiment Prediction Endpoint (`/predict`):**

Accepts JSON-formatted input text(s).

Returns sentiment classification and confidence levels.

Sample Request:

```
POST /predict
{
  "text": "string",
  "texts": [
    "I love this product!",
    "This service is not good."
  ]
}
```

Sample Response:

```
{
  "summary": {
"positive": 0.5023395060561597,
    "negative": 0.481954621442128,
    "neutral": 0.015705934492871165,
    "overall": "positive"
  },
  "comments": [
    {
      "text": "I love this product!",
      "sentiment": "positive",
      "confidence": 0.9942376613616943,
      "all_probs": {
        "positive": 0.9942376613616943,
        "negative": 0.0010972960153594613,
        "neutral": 0.004665091168135405
      }
    },
    {
      "text": "This service is not good.",
      "sentiment": "negative",
      "confidence": 0.9628119468688965,
      "all_probs": {
        "positive": 0.010441350750625134,
        "negative": 0.9628119468688965,
        "neutral": 0.026746777817606926
      }
    }
  ]
}
```

3.5 Frontend Component Structure

Key React components include:

1. **Sentiment Analyzer Component:**

   Manages user inputs and communicates with backend APIs for analysis results.

2. **Sentiment Summary Component:**

   Visualizes sentiment breakdown clearly using interactive graphical elements.

3. **Interactive Charts and Visualizations:**

   Built using Recharts and Framer Motion to deliver an engaging user experience.

3.6 Development Environment Setup

1. **Backend Setup:**

   Python environment setup (`python -m venv venv`, `pip install -r requirements.txt`).

   FastAPI server execution (`uvicorn app:app --reload`).

2. **Frontend Setup:**

   Node.js installation (`npm install`).

   Also set up the following api key and port for backend in .env.local file in the sentiment-analysis-frontend directory

```
NEXT_PUBLIC_YOUTUBE_API_KEY=your_youtube_api_key
NEXT_PUBLIC_BACKEND_URL="http://localhost:8000"
```

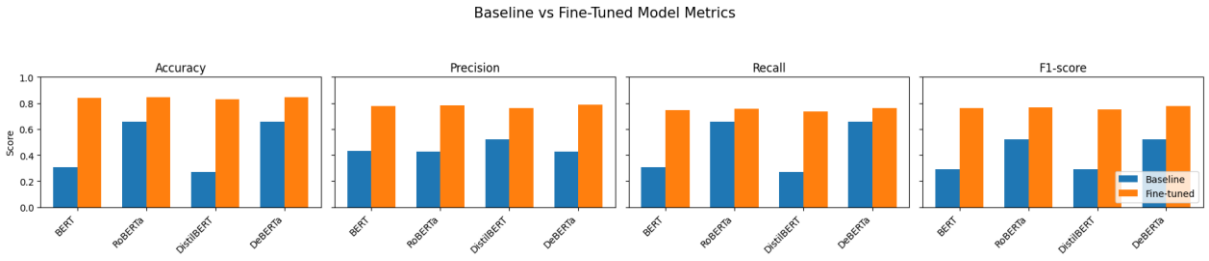
   Next.js application startup (`npm run dev`).

3.7 Model Prediction Logic (DeBERTa)

```
import torch
import torch.nn.functional as F
import numpy as np
from transformers import AutoTokenizer, AutoModelForSequenceClassification

# Device setup
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Load tokenizer and model
model_path = "./deberta_finetuned"
tokenizer = AutoTokenizer.from_pretrained(model_path, local_files_only=True)
model = AutoModelForSequenceClassification.from_pretrained(model_path,
local_files_only=True).to(device)
model.eval()
```

```
# Label mapping
id2label = {0: "positive", 1: "negative", 2: "neutral"}

# Clean input text
def clean_text(t: str) -> str:
    return t.strip()[:512]

# Convert numpy values to native Python types
def convert_numpy(obj):
    if isinstance(obj, np.generic):
        return obj.item()
    elif isinstance(obj, dict):
        return {k: convert_numpy(v) for k, v in obj.items()}
    elif isinstance(obj, list):
        return [convert_numpy(i) for i in obj]
    return obj

# Predict sentiment
def predict_sentiment(texts: list[str], batch_size: int = 32):
    print("Running predict_sentiment on:", len(texts), "texts")

     cleaned_texts = [clean_text(t) for t in texts if isinstance(t, str) and
t.strip()]

    if not cleaned_texts:
        return {
               "summary": {"positive": 0.0, "negative": 0.0, "neutral": 0.0,
"overall": "neutral"},
            "comments": [],
            "aspects": []
        }

    results = []
    total_probs = {"positive": 0.0, "negative": 0.0, "neutral": 0.0}

    for i in range(0, len(cleaned_texts), batch_size):
        batch_texts = cleaned_texts[i:i + batch_size]
            inputs = tokenizer(batch_texts, padding=True, truncation=True,
return_tensors="pt").to(device)

        with torch.no_grad():
            outputs = model(**inputs)
            logits = outputs.logits
            probs = F.softmax(logits, dim=-1).cpu().numpy()

        for text, p in zip(batch_texts, probs):
            max_index = int(p.argmax())
```

```
            sentiment = id2label.get(max_index, "neutral")
            confidence = float(p[max_index])
            result = {
                "text": text,
                "sentiment": sentiment,
                "confidence": confidence,
                "all_probs": {
                    "positive": float(p[0]),
                    "negative": float(p[1]),
                    "neutral": float(p[2])
                }
            }
            results.append(result)
            total_probs["positive"] += float(p[0])
            total_probs["negative"] += float(p[1])
            total_probs["neutral"] += float(p[2])

    n = len(results)
    avg_probs = {
        "positive": total_probs["positive"] / n,
        "negative": total_probs["negative"] / n,
        "neutral": total_probs["neutral"] / n
    }
    overall_label = max(avg_probs, key=avg_probs.get)

    response = {
        "summary": {
            "positive": avg_probs["positive"],
            "negative": avg_probs["negative"],
            "neutral": avg_probs["neutral"],
            "overall": overall_label
        },
        "comments": results,
        "aspects": []
    }

    return convert_numpy(response)
```



For more details on model selection and fine tunning please visit the following notebook on google collab.

https://colab.research.google.com/drive/1MxsmT350vlvPQBBNPAalcVc-xT5_XEBP

3.8 System Advantages

1. **Advanced NLP Models:**
   - DeBERTa transformer models ensure high sentiment classification accuracy.
2. **Real-Time Analytics:**
   - Immediate analysis results, facilitating timely decision-making.
3. **Scalable and Maintainable:**
   - Efficient backend and frontend design ensure ease of maintenance and scalability.
4. **Intuitive User Interface:**
   - User-friendly design ensures accessibility for both technical and non-technical users.

3.9 UML Diagrams

3.9.1 Class Diagram

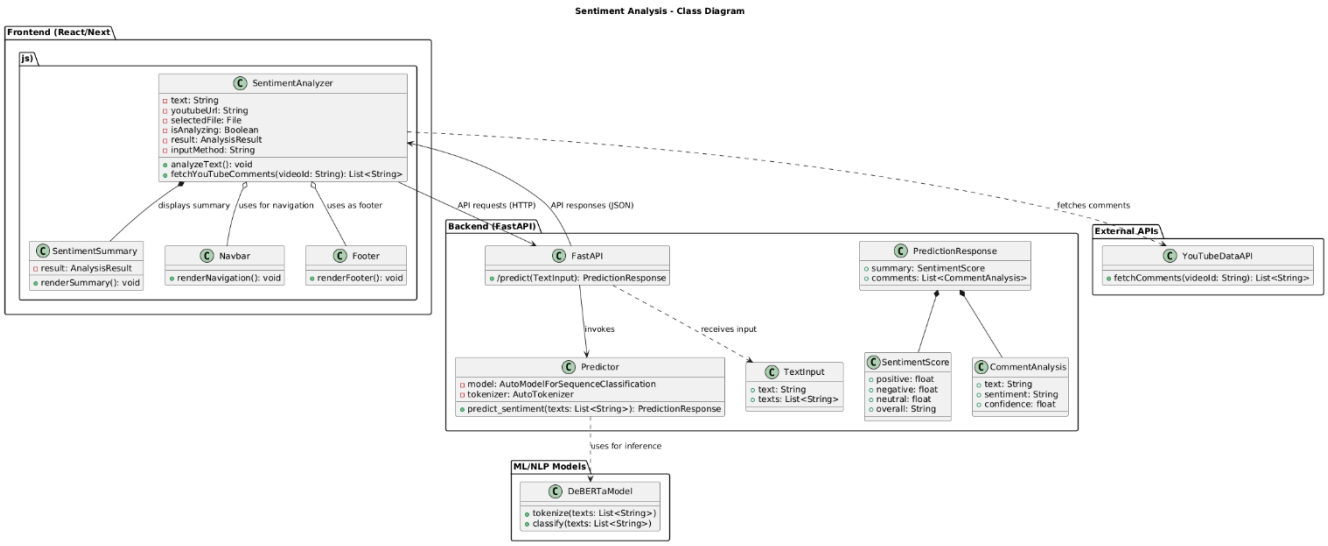


Figure 9: UML Diagram

The following relationships define the structural interactions among various components of the Sentiment Analysis Dashboard:

1. Composition

- **PredictionResponse** is composed of **SentimentScore** and **CommentAnalysis** classes.

SentimentScore and **CommentAnalysis** are tightly integrated components within the PredictionResponse. Without the PredictionResponse class, these components lose their functional context.

- **SentimentAnalyzer** is composed of **SentimentSummary**.

  The SentimentSummary is an integral part of SentimentAnalyzer, specifically designed to visualize results directly linked to the main analyzer functionality.

2. Dependency

- **SentimentAnalyzer** depends on **YouTubeDataAPI**.

  The SentimentAnalyzer component relies on external data retrieval from the YouTubeDataAPI to analyze sentiment from YouTube comments.

- **Predictor** depends on **DeBERTaModel**.

  Predictor utilizes the DeBERTa transformer-based model for accurate sentiment classification, relying on this external NLP library.

- **FastAPI** depends on **TextInput**.

  FastAPI API endpoints use TextInput class for structured input validation, defining the format of received data.

3. Aggregation

- **Frontend UI (SentimentAnalyzer)** aggregates components like **Navbar** and **Footer**.

  Navbar and Footer enhance user interface functionality but can exist independently and are reusable across multiple UI components and pages.

4. Association

- **SentimentAnalyzer** is associated with the backend **FastAPI**.

  The frontend (SentimentAnalyzer) makes HTTP API calls to FastAPI, interacting directly through defined endpoints to exchange sentiment analysis data.

- **FastAPI** is associated with **Predictor**.

  The backend API invokes Predictor methods to process incoming sentiment analysis requests, performing backend computations.

3.9.2 User Case Diagram

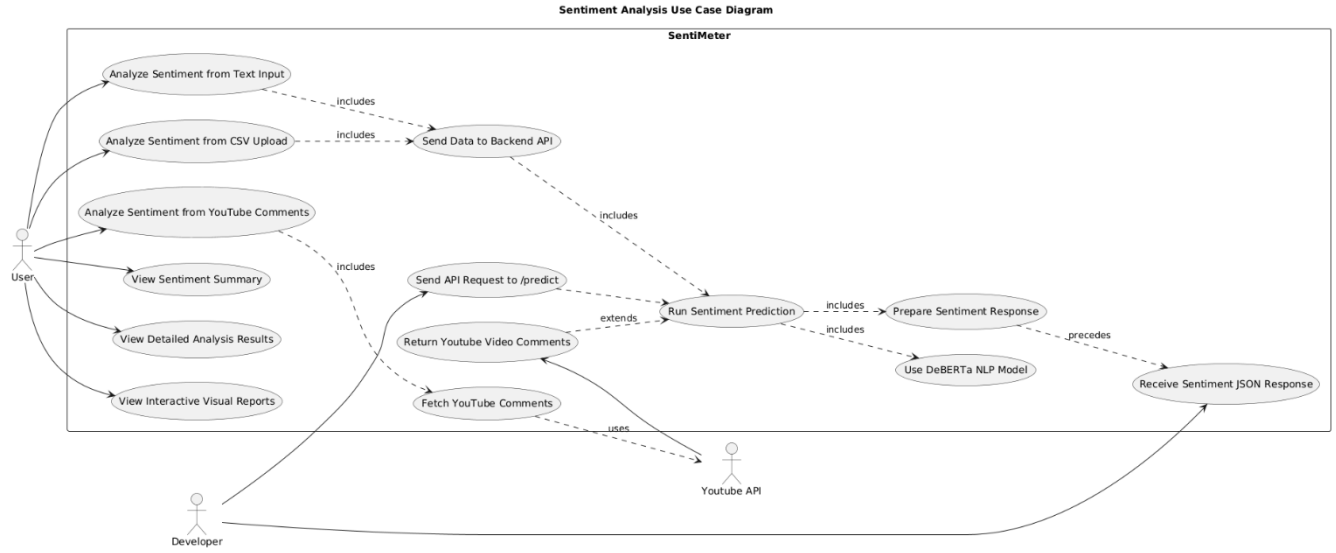


Figure 10: Use Case Diagram

3.9.3 User Sequence Diagram

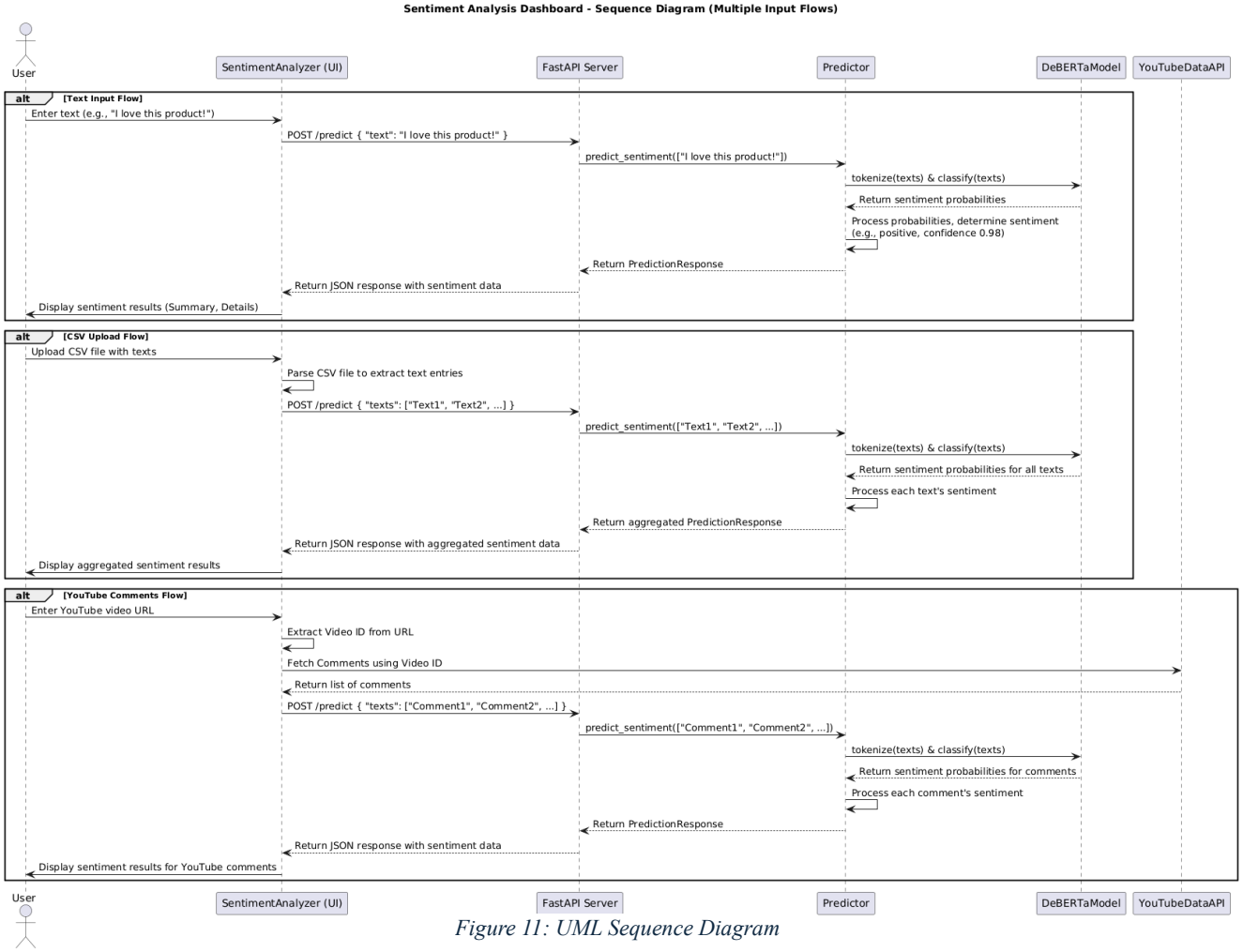


Figure 11: UML Sequence Diagram

3.10 Developer Setup

This section provides a step-by-step guide for setting up the development environment for the Sentiment Analysis Dashboard, covering prerequisites, backend/frontend configuration, and environment variable setup.

3.10.1 Prerequisites

Before setting up the project, ensure the following tools and packages are installed:

- **Python 3.9+** – Required to run the FastAPI backend
- **pip** – Python package manager
- **Node.js (v16+) and npm or yarn** – Required for the frontend (React + Next.js)
- **Git** – For cloning the repository
- **Postman (optional)** – For testing API endpoints

3.10.2 Clone the Repository

1. Open a terminal or command prompt
2. Navigate to the directory where you want to clone the project:

```
cd ~/projects
```

3. Clone the GitHub repository:

```
git clone https://github.com/Malik22G/Thesis.git
cd sentiment-analysis-frontend
```

3.10.3 Setup Python Backend Environment

1. Create a virtual environment:

```
cd Backend/API
```

```
python -m venv venv
```

2. Activate the virtual environment:

- **Linux/macOS:**

```
source venv/bin/activate
```

- **Windows:**

```
venv\Scripts\activate
```

3. Install the required Python dependencies:

```
pip install -r requirements.txt
```

3.10.4 Setup Node.js Frontend Environment

1. Navigate to the frontend folder:

```
cd sentiment-analysis-frontend
```

2. Install frontend dependencies:

```
npm install
```

3. Start the frontend development server:

```
npm run dev
```

3.10.5 Configure Environment Variables

1. In the project root, create a `.env.local` file
2. Add required API keys in the .env.local file:

```
NEXT_PUBLIC_YOUTUBE_API_KEY=your_youtube_api_key
NEXT_PUBLIC_BACKEND_URL="http://localhost:8000"
```

This key is required for the FastAPI backend to access external service YouTube.

The backend_url will run the backend server.

3.11 Interactions and Workflow

This section outlines how different parts of the Sentiment Analysis Dashboard interact throughout the user workflow.

3.11.1 User Interaction Flow

- **Text Input:**

  The user enters raw text in a form

The frontend sends a `POST /predict` request with the text to the backend

The backend processes it and returns sentiment results

- **CSV Upload:**

  The user uploads a CSV file with multiple text entries

  The frontend parses the CSV and sends the content to the backend

  The backend performs sentiment analysis on each entry and returns a list of results

- **YouTube Comment Analysis:**

  The user inputs a YouTube video URL

  The frontend extracts the video ID and fetches top-level comments via YouTube API

  These comments are sent to the backend for sentiment analysis

3.11.2 Backend Processing Flow

- The **FastAPI** server validates the request using `TextInput` (Pydantic)
- It passes the content to the `predict_sentiment()` function in `predictor.py`
- The **DeBERTa model** tokenizes and classifies the texts
- Sentiment predictions are returned along with confidence scores
- A final response includes:
  - Individual comment results
  - An overall sentiment summary

3.11.3 Frontend Display and Feedback

- The frontend receives the structured JSON response from the FastAPI backend, containing both individual sentiment results and an aggregated summary.
- It displays:

Summary metrics, including overall positive, negative, and neutral sentiment percentages.

figure

Detailed results, listing each input (comment or review) along with its predicted sentiment and confidence score.

figure

Top positive and negative texts, allowing users to quickly identify the most emotionally charged comments.

Interactive charts and graphs, rendered using Recharts (e.g., pie charts for sentiment distribution, bar charts for comparison across comments).

- The dashboard uses **Framer Motion** for smooth and responsive animations, ensuring that UI transitions—such as loading states, tab switching, and graph rendering—are visually engaging and modern.
- The layout is **fully responsive**, adapting seamlessly to different screen sizes and devices, making the platform accessible on desktops, tablets, and smartphones.
- Sentiment results are color-coded (e.g., green for positive, red for negative, amber for neutral) to provide visual cues that improve data interpretation at a glance.

3.11.4 Directory Structure

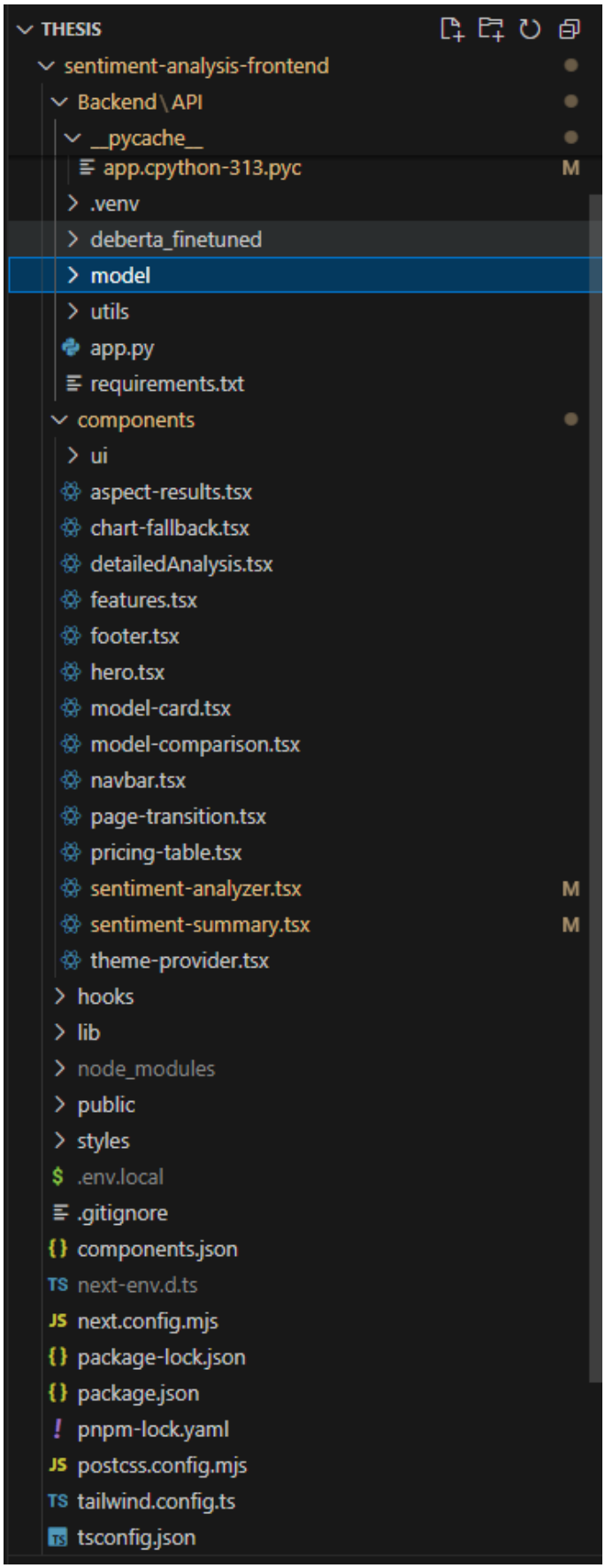


Figure 12: Directory Structure

3.12 Core Components

This section describes the core modules and functionalities implemented in the **Sentiment Analysis Dashboard**. The system is built using modern web technologies such as **FastAPI** for the backend, **React/Next.js** for the frontend, and **DeBERTa** (via Hugging Face Transformers) for the NLP model that powers sentiment prediction.

3.12.1 Sentiment Prediction Module (Predictor)

The `predictor.py` module is the heart of the backend's sentiment analysis functionality. It uses the **DeBERTa transformer model** (fine-tuned for sentiment classification) to categorize input text as **positive**, **negative**, or **neutral**. This is done with high accuracy and contextual understanding.

- **Tokenizer and Model Loading:**

  The model is loaded using `AutoModelForSequenceClassification` and `AutoTokenizer` from Hugging Face.

  The model runs on GPU if available, otherwise falls back to CPU.

- **Key Method – predict_sentiment(texts: List[str]):**

  Preprocesses input texts (trimming and truncating to 512 tokens).

  Runs inference in batches using PyTorch.

  Applies softmax to obtain class probabilities.

  Returns a structured JSON containing:

  A sentiment summary (aggregated score)

  Individual predictions for each text with confidence levels

This module is used by the FastAPI route to handle requests and return consistent output.

3.12.2 FastAPI Server (app.py)

The backend is powered by **FastAPI**, a modern Python web framework used for serving the sentiment analysis API.

TextInput Pydantic Model:

Supports both single `text` and list of `texts` as input.

Validates input before prediction.

POST /predict Endpoint:

Accepts JSON requests for one or more texts.

Routes the request to the predictor for sentiment classification.

3.12.3 Frontend Interface (React + Next.js)

The frontend of the system is implemented using **React** with **Next.js**, and styled using **Tailwind CSS**. The UI is interactive, responsive, and animated using **Framer Motion**.

- **SentimentAnalyzer Component:**

  Central component handling user input and communication with the backend.

  Allows three input types:

  1. **Text**: Single text entry for analysis.
  2. **CSV Upload**: For bulk feedback analysis (parsed client-side).
  3. **YouTube URL**: Fetches video comments and analyzes sentiment.
- **SentimentSummary Component:**

  Displays overall sentiment breakdown (positive/negative/neutral percentages).

  Uses Recharts for animated bar and pie graphs.

- **Tabs and Animations:**

  React Tabs and Framer Motion animate switching between summary and detailed views.

  Sentiment results are visualized with colored progress bars and labeled segments.

3.12.4 YouTube Comment Integration

The system supports sentiment analysis of **YouTube comments** through client-side interaction with the **YouTube Data API v3**.

- **Video ID Extraction:**

Parses both standard and shortened YouTube URLs.

- **Comment Fetching:**

  Makes a fetch call to the YouTube API (via `fetchYouTubeComments`) using the video ID and API key.

  Extracts the top-level comments for sentiment processing.

3.12.5 File Upload and CSV Processing

The platform allows CSV uploads (containing text entries) for batch sentiment analysis.

- **Upload Handling:**

  Handled in the frontend using an `<input type="file">` control.

- **Client-side Parsing:**

  Currently simulated with placeholder data but designed to accept and parse actual CSV content.

- **Backend Communication:**

  Text entries are sent as a `texts` list to the `/predict` endpoint for batch classification.

3.12.6 Supporting Utilities and Components

- **Navbar / Footer Components:**

  Provide consistent layout and navigation across pages.

- **Progress and Card UI:**

  Cards show sentiment per text with visual indicators (thumbs up/down/neutral).

  Colored progress bars visually represent sentiment distribution.

3.13 Testing

To ensure the reliability, correctness, and stability of the **Sentiment Analysis Dashboard**, two key testing tools were utilized throughout development:

- **Pytest**: For validating backend logic, model inference, and handling of different input formats.
- **Postman**: For API endpoint testing, response structure validation, and end-to-end request simulation.

3.13.1 Testing using Pytest

Pytest was used to perform unit and functional tests on the FastAPI backend. Key areas tested include sentiment prediction logic, response formatting, error handling, and data validation.

test_predictor.py

- **test_single_text_prediction()**

  Purpose: Validates sentiment prediction for a single user-provided text.

  Assertions: Ensures the response includes a sentiment label (positive/negative/neutral) and a valid confidence score.

- **test_batch_text_prediction()**

  Purpose: Tests handling of multiple text inputs in a batch request.

  Setup: Sends a list of comments to /predict.

  Assertions: Confirms correct structure and classification for each item.

- **test_empty_input()**

  Purpose: Verifies the system handles empty or invalid input gracefully.

  Assertions: Checks that an error or default response is returned without crashing.

- **test_invalid_data_type ()**

  Purpose: Ensures input with invalid data types (e.g., None, numbers, objects) is ignored safely without causing a crash.

  Assertions: Confirms that only valid string entries are processed, and the function returns a correct result without raising an exception.

- **test_sentiment_confidence_bounds ()**

  Purpose: Validates that confidence values are within [0.0, 1.0].

  Assertions: All confidence scores in output are bounded correctly.

- **test_summary_probabilities_sum_to_one ()**

  Purpose: Verifies the system handles empty or invalid input gracefully.

  Assertions: Checks that an error or default response is returned without crashing.

- **test_mixed_sentiments_batch ()**

  Purpose: Tests classification variety across diverse sentiments.

  Assertions: At least two different sentiment labels appear in result.

- **test_long_text_truncation ()**

  Purpose: Verifies long text inputs are truncated safely.

  Assertions: No crash; sentiment is predicted.

```
import numpy as np
import pytest
from predictor import predict_sentiment

def test_single_text_prediction():
    input_text = "I absolutely love this product!"
    result = predict_sentiment([input_text])

    assert "summary" in result
    assert "comments" in result
    assert len(result["comments"]) == 1

    comment_result = result["comments"][0]
    for key in ["text", "sentiment", "confidence", "all_probs"]:
        assert key in comment_result

    assert comment_result["sentiment"] in {"positive", "negative", "neutral"}
    assert isinstance(comment_result["confidence"], float)
    assert 0.0 <= comment_result["confidence"] <= 1.0

def test_batch_text_prediction():
    input_texts = [
      "This is amazing, I love it!",
      "Worst experience ever. Totally disappointed.",
      "It's okay, not great but not terrible."
    ]
    result = predict_sentiment(input_texts)
```

```
    assert "comments" in result
    assert len(result["comments"]) == len(input_texts)

    for comment in result["comments"]:
         for key in ["text", "sentiment", "confidence", "all_probs"]:
             assert key in comment

def test_empty_input():
    result = predict_sentiment([])
    assert "summary" in result
    summary = result["summary"]
    assert summary["positive"] == 0.0
    assert summary["negative"] == 0.0
    assert summary["neutral"] == 0.0
    assert summary["overall"] == "neutral"
    assert result["comments"] == []
    assert result["aspects"] == []


def test_invalid_data_type():
    input_texts = [None, 12345, {"text": "invalid"}, "Valid comment"]
    result = predict_sentiment(input_texts)

    # Only the last valid string should be processed
    assert len(result["comments"]) == 1
    assert result["comments"][0]["text"] == "Valid comment"
      assert  result["comments"][0]["sentiment"]  in  {"positive",  "negative",
"neutral"}


def test_sentiment_confidence_bounds():
    result = predict_sentiment(["Nice product", "Terrible service"])
    for comment in result["comments"]:
        assert 0.0 <= comment["confidence"] <= 1.0

def test_summary_probabilities_sum_to_one():
    result = predict_sentiment(["Amazing", "Horrible", "Meh"])
      total  =  sum(result["summary"][s]  for  s  in  ["positive",  "negative",
"neutral"])
    assert abs(total - 1.0) < 0.01

def test_mixed_sentiments_batch():
    texts = ["Great!", "Awful!", "Okay."]
    result = predict_sentiment(texts)
    sentiments = {c["sentiment"] for c in result["comments"]}
    assert len(sentiments) >= 2

def test_long_text_truncation():
```

```
    long_text = "excellent " * 200
    result = predict_sentiment([long_text])
      assert  result["comments"][0]["sentiment"]  in  {"positive",  "negative",
"neutral"}
```

```
(venv) PS C:\Users\User\Desktop\Thesis\sentiment-analysis-frontend\Backend\API> pytest .\model\test_predictor.py
====================== test session starts ======================
platform win32 -- Python 3.13.3, pytest-8.2.1, pluggy-1.5.0
rootdir: C:\Users\User\Desktop\Thesis\sentiment-analysis-frontend\Backend\API
plugins: anyio-4.9.0
collected 8 items

model\test_predictor.py ........                          [100%]

======================= 8 passed in 14.28s ======================
(venv) PS C:\Users\User\Desktop\Thesis\sentiment-analysis-frontend\Backend\API>
```

test_csv_upload.py

- **test_parse_csv()**

  Purpose: Tests parsing logic for a simulated CSV upload.

  Assertions: Ensures correct conversion of CSV rows to text input for analysis.

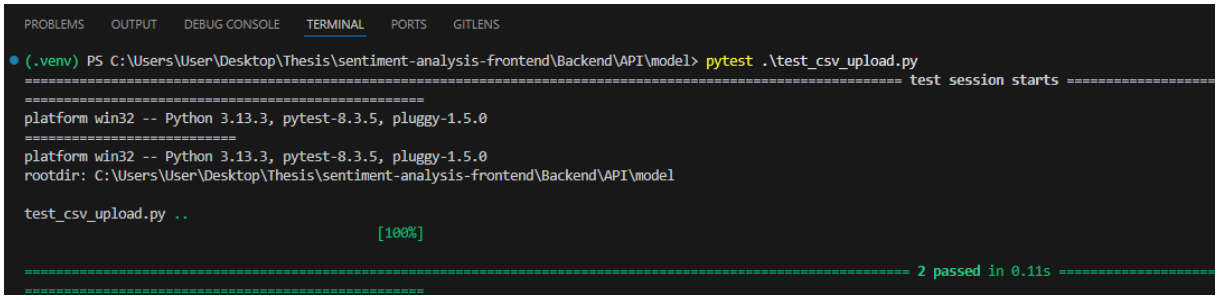
- **test_large_csv_batch()**

  Purpose: Validates performance and correctness on a large number of comments.

  Assertions: Checks response size and accuracy under higher volume.

```
import io
import csv

def parse_csv(file_like_object):
    reader = csv.reader(file_like_object)
    return [row[0] for row in reader if row]

def test_parse_csv():
    csv_content = "This is comment one\nThis is comment two\nThis is comment
three"
    file_like = io.StringIO(csv_content)
    texts = parse_csv(file_like)
```

```
    assert len(texts) == 3
    assert texts[0] == "This is comment one"
    assert texts[1] == "This is comment two"
    assert texts[2] == "This is comment three"

def test_large_csv_batch():
    comment = "This is a test comment."
    num_rows = 1000
    csv_content = "\n".join([comment for _ in range(num_rows)])
    file_like = io.StringIO(csv_content)
    texts = parse_csv(file_like)
    assert len(texts) == num_rows
    for text in texts:
        assert text == comment
```



3.13.2 API Testing Using Postman

Postman was used extensively during development to test and verify API functionality, including input/output behavior, error handling, and response structure for different request types.

Prerequisites

- **API Running Locally**: FastAPI server running at http://13.61.50.162:8000
- **Postman Installed**: Download from https://www.postman.com/downloads/

Base URL

http://13.61.50.162:8000

3.13.2.1 Prediction Endpoints

1. Analyze Single Text

- **Endpoint:** /predict

- **Method:** POST
- **Headers:** Content-Type: application/json
- **Body:**

```
{
  "text": "The service was excellent and fast!"
}
```

- **Expected Response:**
  - JSON object with sentiment, confidence, and summary.

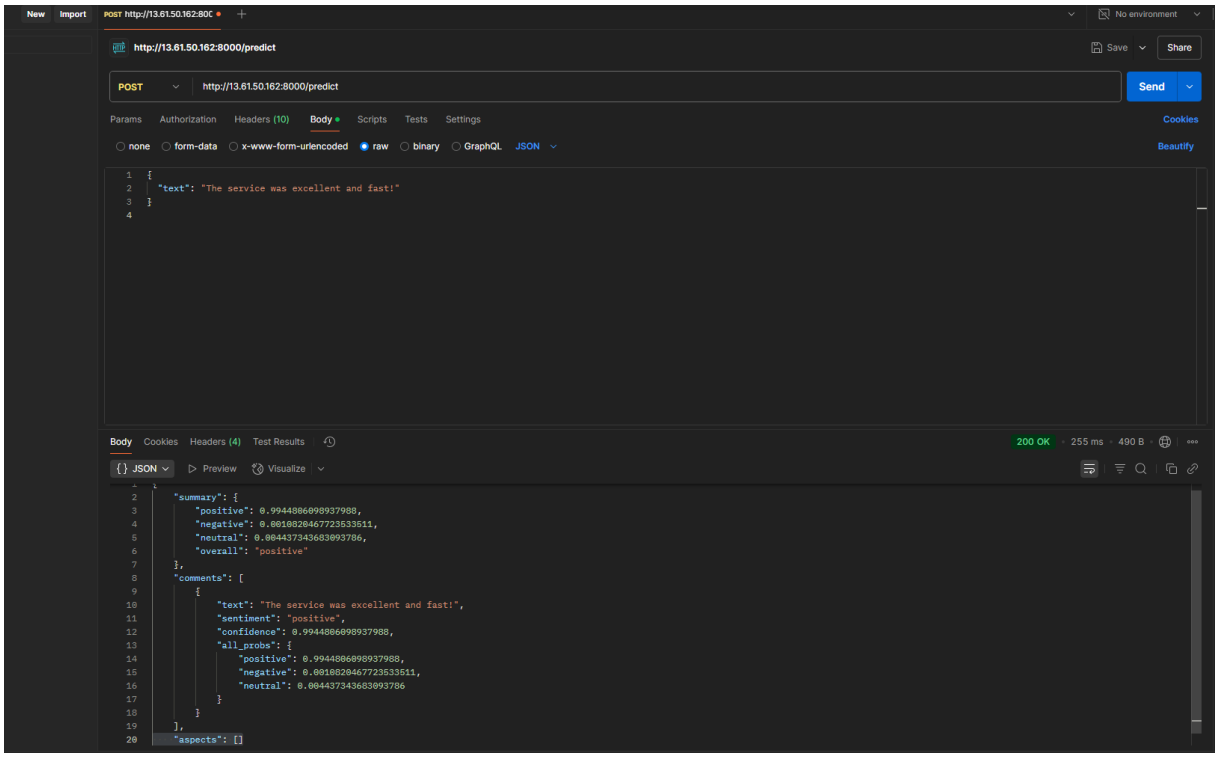


Figure 13: Using Postman to test API for single text input

2. Analyze Multiple Texts (Batch/CSV Upload)

- **Endpoint:** /predict
- **Method:** POST
- **Headers:** Content-Type: application/json
- **Body:**

```
{
  "texts": ["Loved the app!", "This was disappointing.", "I wish I had worked hard."]
}
```

- **Expected Response:**
  - List of sentiment results with scores and labels, including aggregated summary.

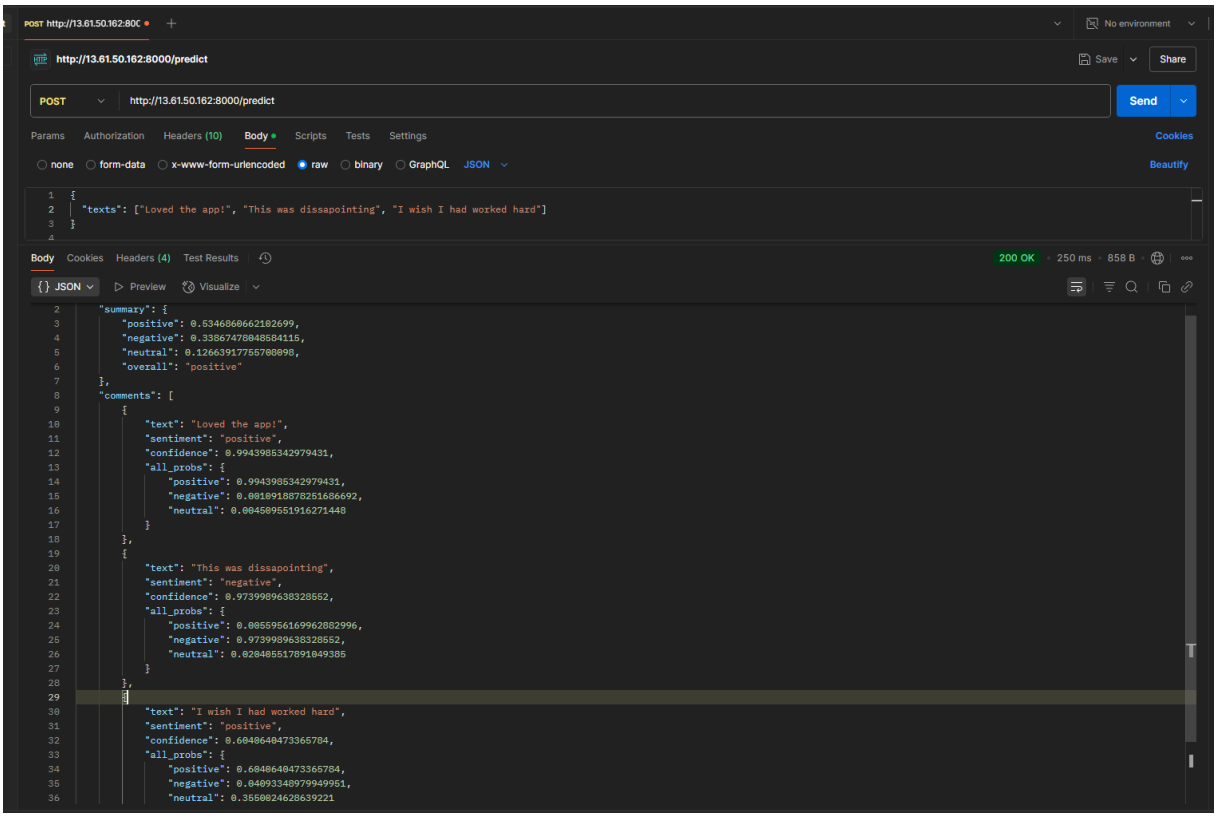


Figure 14: Testing API for multiple texts in case of CSV

3. Analyze YouTube Comments

- **Frontend Flow:**

  The YouTube video URL is processed client-side to extract the videoId, then the system fetches up to 100 comments using the YouTube API.

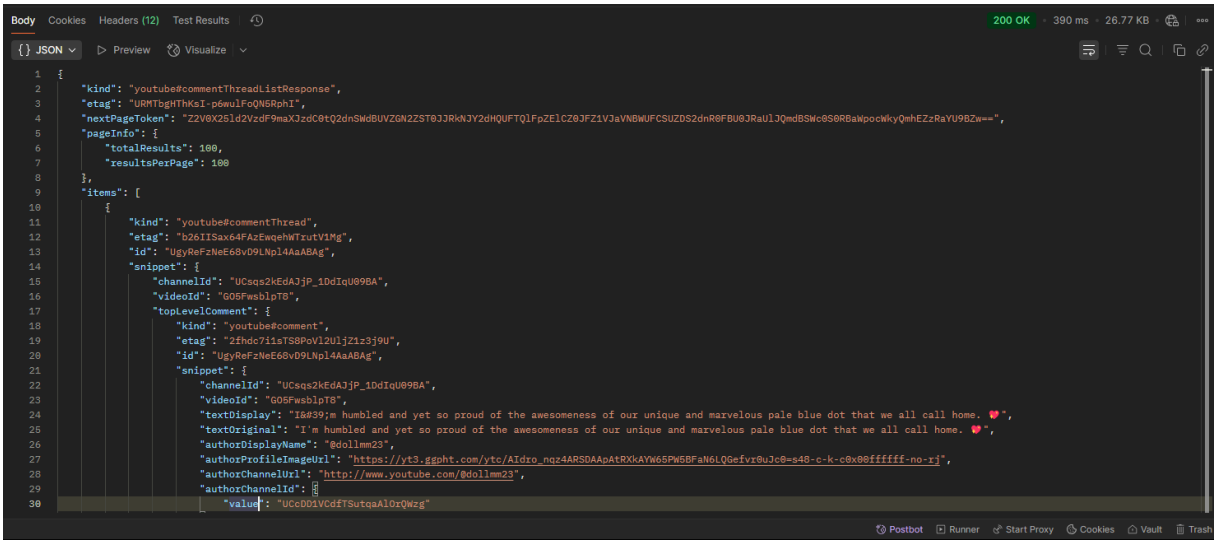


Figure 15: Response from YouTube API

- **Back-end Interaction:**

  Extracted comments are sent to the /predict endpoint for batch analysis.

- **Expected Output:**

  Response includes sentiment for each comment and an overall sentiment summary.

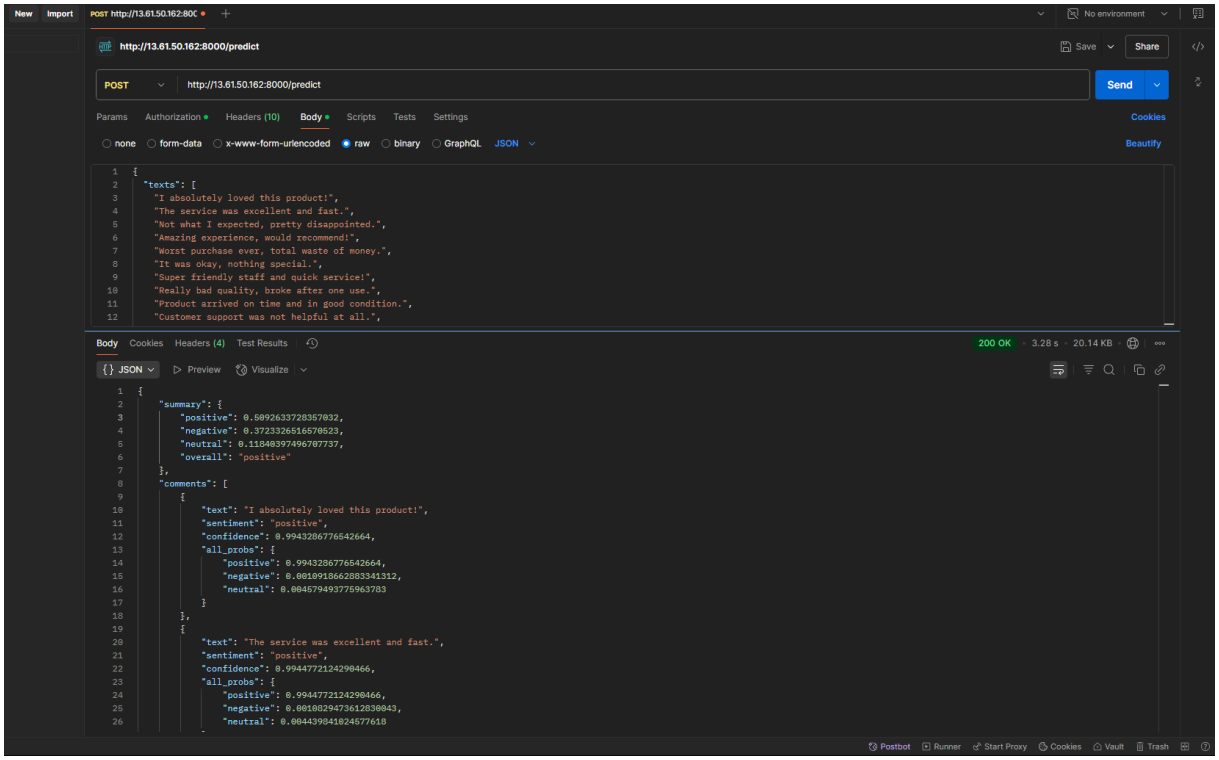


Figure 16 Testing API for multiple texts in case of YouTube Comments

3.8.2.2 Error & Validation Tests

1. Empty Input

- **Body:**

  {}

- **Expected:**

  Error message: "No input text provided."

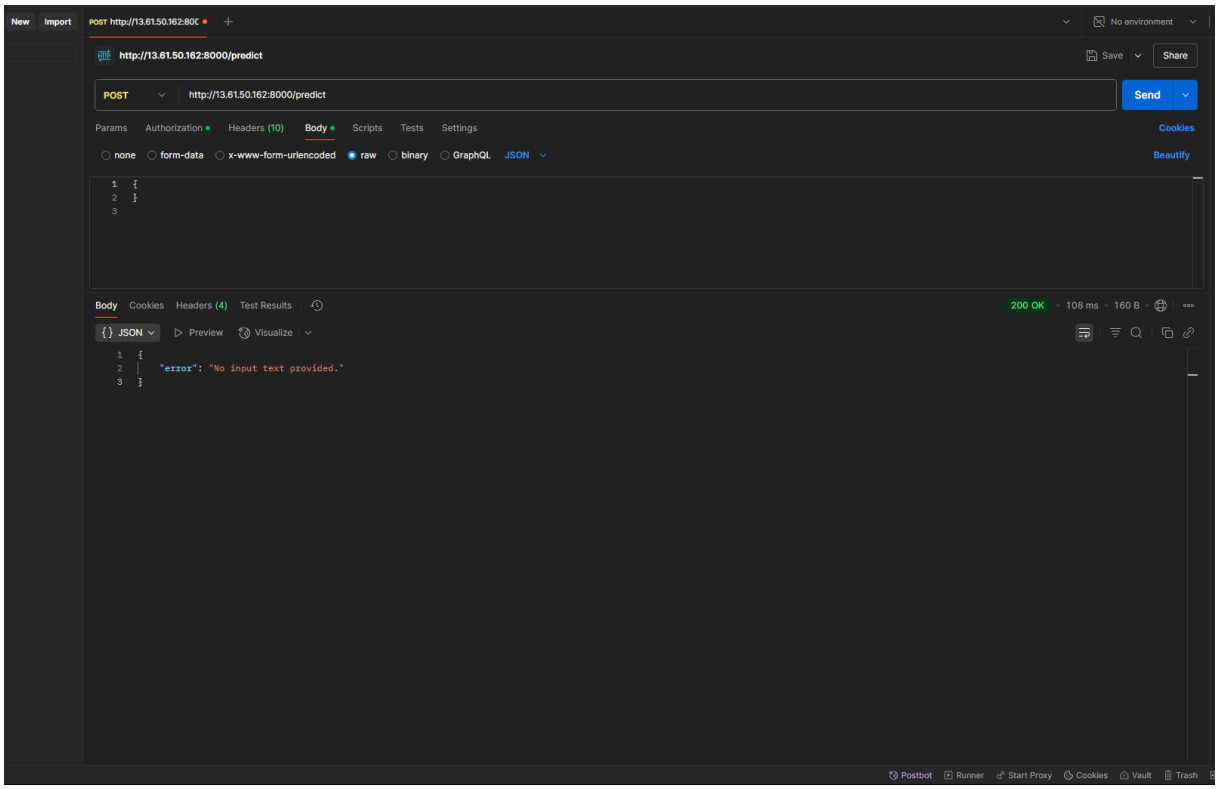


Figure 17: Empty request body response Test

Chapter 4

4. Conclusion and future work

4.1 Conclusion

The **Sentiment Analysis Dashboard** successfully delivers a robust and user-friendly platform for performing real-time sentiment classification on a wide variety of textual data inputs. Users can analyze sentiment through three intuitive methods: direct text input, batch CSV uploads, and automated extraction of YouTube video comments.

The backend, developed using **FastAPI** and powered by the **DeBERTa transformer model**, offers accurate and efficient classification into positive, negative, and neutral sentiments. The frontend, built with **React, Next.js**, and styled using **Tailwind CSS**, presents results in a clear and interactive manner through dynamic charts and animations powered by **Recharts** and **Framer Motion**.

The system's support for modern NLP models, real-time visualization, and multi-format input capabilities ensures that both technical and non-technical users can derive meaningful insights from raw text data. Overall, the application provides a scalable and effective solution for real-world sentiment analysis tasks.

4.2 Future Work

To further enhance the functionality, user engagement, and analytical capabilities of the platform, the following improvements are proposed for future development:

1. **Sentiment Export Functionality**

   Allow users to export sentiment results (as PDF or CSV) for use in reports, research, or business presentations.

2. **User Authentication and History**

   Enable account creation and login features to allow users to save and revisit previous analysis sessions.

3. **Comment-Level Sentiment Highlighting**

Display sentiment annotations directly within the analyzed text to help users understand emotional shifts across comments or documents.

4. **Multilingual Support**

   Integrate language detection and model-switching to support sentiment analysis across different languages.

5. **Admin Dashboard for Analytics**

   Develop a backend interface to monitor usage statistics, model performance, and API logs for maintenance and insight.

6. **Advanced Visualization & Filtering**

   Add filters for viewing sentiment by time, topic, or source, and include advanced visualization types like heatmaps or word clouds.

7. **Integrate Real-Time APIs (e.g., Twitter/X)**

   Expand data input sources to include live streams from Twitter or Reddit for sentiment monitoring in social media.

By implementing these future enhancements, the **Sentiment Analysis Dashboard** can evolve into a full-featured, intelligent analysis suite for businesses, researchers, and content creators alike—pushing the boundaries of accessible and powerful natural language understanding.

Bibliography

1. **Vaswani, A., et al.** (2017). *Attention is All You Need*. Retrieved from https://arxiv.org/abs/1706.03762
2. **Microsoft Research**. (2025). *DeBERTa: Decoding-enhanced BERT with Disentangled Attention*. Retrieved from https://github.com/microsoft/DeBERTa
3. **Hugging Face**. (2025). *Transformers Documentation*. Retrieved from https://huggingface.co/docs/transformers
4. **Python Software Foundation**. (2025). *Python 3.12 Documentation*. Retrieved from https://docs.python.org/3.12/
5. **FastAPI Developers**. (2025). *FastAPI: High Performance, Easy to Learn, Fast to Code, Ready for Production*. Retrieved from https://fastapi.tiangolo.com/
6. **Pydantic Developers**. (2025). *Pydantic v2 – Data validation and settings management*. Retrieved from https://docs.pydantic.dev/latest/
7. **React Developers**. (2025). *React – A JavaScript Library for Building User Interfaces*. Retrieved from https://react.dev/
8. **Vercel**. (2025). *Next.js Documentation*. Retrieved from https://nextjs.org/docs
9. **Tailwind Labs**. (2025). *Tailwind CSS v3.4 Documentation*. Retrieved from https://tailwindcss.com/docs
10. **Framer**. (2025). *Framer Motion – Production-Ready Motion Library for React*. Retrieved from https://www.framer.com/motion/
11. **Recharts**. (2025). *Recharts – React Charting Library*. Retrieved from https://recharts.org/
12. **Google Developers**. (2025). *YouTube Data API v3 Documentation*. Retrieved from https://developers.google.com/youtube/v3
13. **Postman**. (2025). *Postman: The Complete API Development Platform*. Retrieved from https://www.postman.com/
14. **GitHub**. (2025). *GitHub – The Complete Platform for Developers*. Retrieved from https://github.com/
15. **PyTorch**. (2025). *PyTorch – Deep Learning Framework*. Retrieved from https://pytorch.org/docs/stable/index.html

List of Figures

Acknowledgment

First and foremost, I would like to express my heartfelt gratitude to my supervisor, **Md Easin Arafat**, for his unwavering support and invaluable guidance throughout the course of this project. His deep expertise in the field, constructive feedback, and consistent encouragement have been instrumental in shaping the direction and quality of this thesis. His dedication to mentoring and high standards of academic excellence have continuously inspired me to improve and strive for the best.

I would also like to extend my sincere thanks to my family for their unconditional love, endless patience, and constant support. Their belief in me, especially during moments of challenge and doubt, has been the foundation of my resilience and determination. I am truly grateful for their presence, motivation, and sacrifices which have empowered me to successfully complete this academic endeavor.

This work is a reflection of the collective support, encouragement, and inspiration I have received from those around me. I am deeply thankful to everyone who played a role in this journey.